Projet « Moteur de recherche » — Partie 1 sur 3

Acquisition de données, conception et structuration du code en POO

Programmation de spécialité (Python) (≈ 5–6 h)

Julien Velcin* Walid Bechkit†

Année universitaire 2026–2027

L'objectif de ce projet, réparti en **trois parties**, est de mettre au point un **moteur de recherche d'information « maison »**. Il ne s'agit donc pas d'utiliser l'une des (nombreuses) librairies existantes, mais d'implémenter sa *propre solution*. C'est l'occasion de mettre en œuvre les connaissances du cours tout en acquérant une meilleure compréhension des techniques souvent cachées derrière les librairies « prêtes à l'emploi » (par ex. `scikit-learn` ou `nltk`) : leur usage est donc **interdit** pour le cœur du moteur.

Cette **première partie** couvre l'*acquisition* des données puis la *conception* et la *structuration* du code à l'aide de classes et de modules (paradigme objet). Elle correspond à environ 5 à 6 heures de travail et se déroule en plusieurs séances. Les librairies d'*acquisition* (`praw`, `urllib`, `xmltodict`) et de manipulation de données (`pandas`) sont, elles, autorisées.

Partie 1 : Acquisition des données

Cette première étape consiste à extraire des documents à partir de sources externes. Ces documents s'organisent en une collection appelée *corpus*. On considère plusieurs sources d'information que l'on souhaite croiser sur un sujet de votre choix, que nous appellerons **thématique**. Cette thématique doit pouvoir être identifiée simplement par un ou des mots-clés, par exemple : « Coronavirus », « football », « climate » (nous travaillerons en anglais).

Nous allons d'abord récupérer le contenu textuel. Commencez par créer une liste *docs* : chaque entrée sera un document (donc, pour le moment, un simple texte). Les documents viendront de *deux* sources : **Reddit** et **Arxiv**. Vous les récupérerez à l'aide de mots-clés liés à votre thématique, comme indiqué ci-dessous.

1.1 Reddit. À l'aide de la librairie `praw`, interrogez l'API comme expliqué sur ce site web. Quels sont les champs disponibles ? Quel est le champ contenant le contenu textuel ? Alimentez la liste *docs* avec ce texte uniquement. Vous pouvez déjà vous débarrasser des sauts de ligne (`\n`) en les remplaçant par des espaces.

*Auteur du sujet original.

†Responsable du cours

1.2 Arxiv. À l'aide de la librairie `urllib`, interrogez l'API d'Arxiv en vous aidant de l'aide en ligne. Parsez les résultats grâce à la librairie `xmltodict`. Quels sont les champs disponibles ? Quel est le champ contenant le contenu textuel ? Alimentez la liste *docs*.

Partie 2 : Sauvegarde et rechargement des données

L'objectif est de ne pas avoir à interroger les API à chaque exécution du programme.

2.1 Créez un tableau de type `DataFrame` de la librairie `pandas` et instanciez-le avec les textes récupérés depuis les API. Vous utiliserez trois colonnes : un **identifiant unique** du texte (par ex. un simple entier naturel), le **texte**, et son **origine** (`reddit` ou `arxiv`).

2.2 Sauvegardez ce tableau sur le disque dans un fichier `.csv` en utilisant la tabulation `\t` comme séparateur.

2.3 Ajoutez le code permettant de recharger directement ce tableau en mémoire, sans repasser par l'interrogation des API.

Partie 3 : Premières manipulations du corpus

Nous allons manipuler simplement notre corpus.

3.1 Affichez la taille de votre corpus, c'est-à-dire le nombre de documents.

3.2 Pour chaque document, affichez le nombre de **mots** et de **phrases**. Vous utiliserez la fonction `split` en considérant que les mots sont séparés par des espaces et les phrases par des points. Il s'agit bien sûr d'une simplification ; des techniques plus avancées existent en TAL (traitement automatique du langage).

3.3 Supprimez de votre corpus les documents trop petits, ici ceux qui contiennent moins de 100 caractères.

3.4 Créez une unique chaîne de caractères contenant tous les documents grâce à la fonction `join`. Cette chaîne vous sera utile dans la suite du projet.

Partie 4 : Première classe : le `Document`

Le contenu textuel n'est pas la seule ressource importante du corpus. Les documents sont souvent associés à de nombreuses *méta-données* : nom de l'auteur, année de publication, tags, etc. Pour les gérer, nous passons au paradigme objet.

4.1 Dans un fichier (module) à part, intitulé `Document.py`, créez une classe `Document` contenant les attributs suivants :

- *titre* : le titre du document ;
- *auteur* : le nom de l'auteur ;
- *date* : la date de publication ;
- *url* : l'URL source ;
- *texte* : le contenu textuel du document.

4.2 Créez une première méthode qui affiche toutes les informations d'une instance. Implémentez ensuite la méthode `__str__(self)`, qui affichera une version « digeste » de l'instance (par exemple son titre seulement). Cette méthode est appelée lorsque vous exécutez `print(doc)`, où `doc` est une instance de `Document`.

4.3 Modifiez le code de la partie précédente afin d'instancier des objets `Document`. Attention à bien **importer** le module dans le fichier principal. Pour la date de publication, vous pouvez la traiter comme une chaîne `str`, mais le mieux est de passer par la librairie `datetime`.

Il faut créer une **collection unique** contenant toutes les instances de documents, quelle que soit leur origine (Reddit ou Arxiv). Au lieu d'une simple liste, vous utiliserez un **dictionnaire** Python indexant chaque document par un identifiant numérique unique (généré séquentiellement). Vous appellerez ce dictionnaire *documents* : les clés sont les identifiants uniques (`int`), les valeurs sont les instances de `Document` correspondantes.

Partie 5 : Gestion des auteurs : la classe `Author`

Les auteurs aussi portent des méta-données intéressantes. Vous allez implémenter une classe pour les gérer.

5.1 Créez une classe `Author` contenant les attributs suivants :

- `name` : son nom ;
- `nb_docs` : le nombre de documents publiés ;
- `production` : un dictionnaire des documents écrits par l'auteur.

Mettez cette classe dans un fichier (module) à part, intitulé `Author.py`.

5.2 Créez une méthode `add`, propre à la classe `Author`, qui alimente l'attribut `production` (elle compile l'ensemble des documents rédigés par l'auteur). Implémentez ensuite la méthode `__str__(self)`.

5.3 Dans le fichier principal, créez un dictionnaire *authors* contenant les auteurs, avec pour clé leur nom (considéré comme unique). Pour l'alimenter, ajoutez une instance d'auteur chaque fois que vous rencontrez un **nouvel** auteur : vous devrez donc vérifier au préalable si son nom fait déjà partie des clés du dictionnaire.

5.4 Ajoutez le code permettant d'obtenir quelques statistiques sur un auteur saisi par l'utilisateur : nombre de documents produits et taille moyenne des documents.

Partie 6 : La classe `Corpus`

Il s'agit maintenant de structurer le code du fichier principal à l'intérieur d'une classe dédiée à la gestion d'un corpus. Cela permet de gérer *plusieurs* corpus, ou un corpus qui évolue dans le temps.

6.1 Créez une classe `Corpus`, dans son propre fichier `Corpus.py`, à partir des méthodes d'ajout des parties précédentes, et contenant les attributs suivants :

- *nom* : le nom du corpus ;
- *documents* : le dictionnaire des documents ;
- *id_ document* : le compteur de documents ;
- *authors* : le dictionnaire des auteurs.

6.2 Implémentez des méthodes permettant d'afficher les éléments du corpus triés selon la **date**, puis selon le **titre**. Ces méthodes prennent en paramètre le nombre de documents à afficher. Implémentez également la méthode `__repr__(self)` pour un affichage plus digeste.

6.3 Créez enfin des méthodes *save* et *load* pour enregistrer et charger le corpus sur le disque. Pour cela, enrichissez le code de la partie précédente afin de passer par une table `DataFrame`. D'autres formats sont bien sûr possibles (`pickle`, `json`) : n'hésitez pas à les tester (optionnel).

6.4 Sauvegardez puis rechargez vos données pour vous assurer que toute la chaîne de traitement est opérationnelle.

Partie 7 : Héritage : `RedditDocument` et `ArxivDocument`

Les deux types de documents intégrés au corpus (*posts* Reddit et articles Arxiv) possèdent de nombreux points communs (titre, auteur, date, etc.) mais aussi des différences. Par exemple, les posts Reddit déclenchent des commentaires dont l'API permet de récupérer le nombre ; les articles Arxiv sont souvent écrits par *plusieurs auteurs*, dont on peut récupérer la liste. On vous demande de mettre en place un mécanisme d'**héritage** en implémentant deux classes filles. On conseille de les écrire dans le fichier `Document.py`, à la suite de la classe mère.

7.1 Créez une classe fille `RedditDocument` qui hérite de `Document`. Elle doit intégrer au moins une variable spécifique aux documents Reddit, absente des autres sources (par exemple le nombre de commentaires). Vous implémenterez notamment :

- un **constructeur**, en pensant à appeler celui de la classe mère (si possible via le mot-clé `super()`) ;
- des **accesseurs/mutateurs** pour les champs spécifiques ;
- une méthode `__str__` spécifique pour l'affichage de l'objet.

7.2 De la même manière, créez une classe fille `ArxivDocument` qui gère la liste des **co-auteurs** des articles comme élément spécifique.

Partie 8 : Polymorphisme et type de document

8.1 Testez votre objet `Corpus` en y ajoutant des objets des deux classes que vous venez de créer. Le **polymorphisme** (la capacité d'une variable, d'une fonction ou d'un objet à prendre plusieurs formes) doit vous permettre de réaliser cette opération *sans modifier* les méthodes de la classe `Corpus`.

8.2 Vous devez pouvoir afficher la liste des articles du corpus avec leur *source* (Reddit ou Arxiv). Parmi plusieurs solutions possibles, implémentez celle-ci : ajoutez un champ `type` à la classe mère `Document`, et implémentez la méthode `getType()` directement dans les classes filles, comme vu en cours.

Partie 9 : Patrons de conception : `Singleton` et *Factory*

9.1 Pour le moment, on ne traite qu'un **unique** corpus. Profitez-en pour mettre en œuvre un patron de type `Singleton` garantissant cette unicité.

9.2 Créez un **générateur de documents** à l'aide d'un patron d'usine (*factory pattern*) : selon la source (Reddit ou Arxiv), la fabrique renverra une instance de la classe fille appropriée.